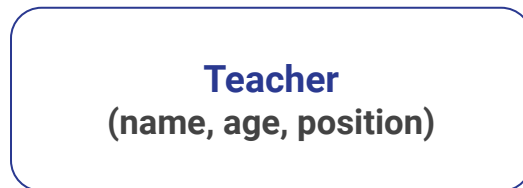
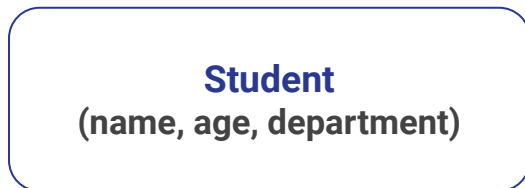


Outlines

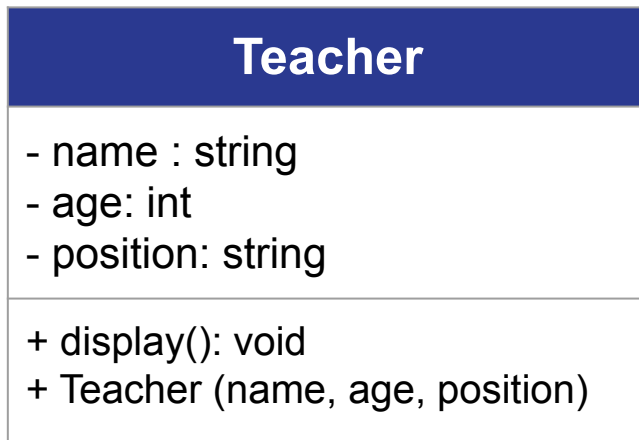
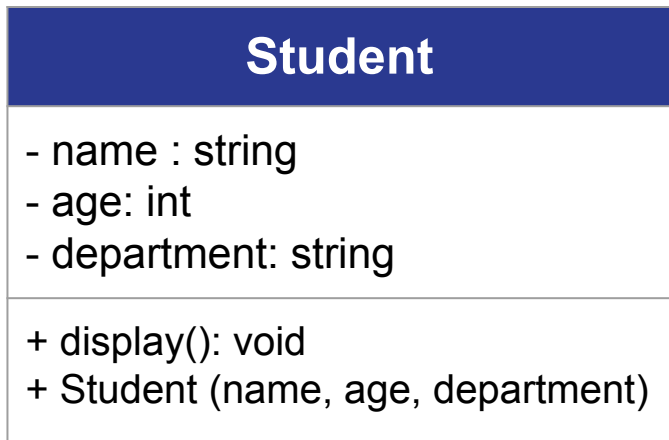
- Introduction
- Base and Derived class
- Access Specifiers - Protected
- Visibility Modes
- Types of Inheritance
- Virtual Base Class
- Function Overriding
- Abstract Class
- Constructor and Destructor

Example

Assume two classes:



Class Diagram:



Access symbols

- private

+ public

protected

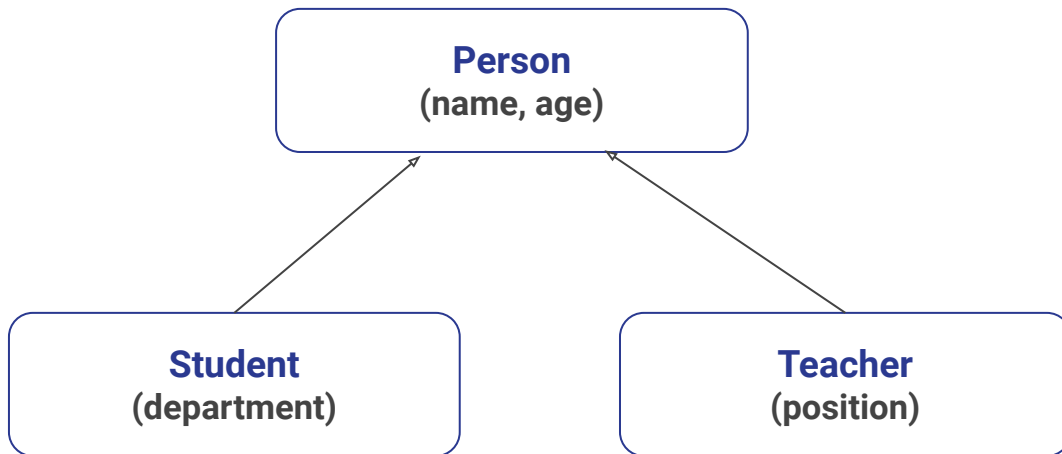
Problem

Both the **Student** and **Teacher** classes have attributes for **name** and **age**, leading to redundancy in code.

Duplication of **name** and **age** attributes violates the principle of code reusability and maintainability.

Solution- Using Inheritance

Utilize inheritance to create a base class that encapsulates common attributes.



Benefits:

- Promotes code reusability.
- Improves maintainability.

Introduction

Inheritance is one of the key features of Object-oriented programming in C++. It allows us to create a new class (derived class) from an existing class (base class).

Inheritance allows one class to acquire the properties and behaviors of another class.

The **derived** class inherits the features from the **base** class and can have additional features of its own.

Purpose: Promotes code reusability and establishes a relationship between classes.

- Inheritance in C++ takes place between classes.
- A child class inherits both behaviors (member functions) and properties (member variables) from the parent (with different access permission).
- These variables and functions become members of the derived class.
- Child classes can have their own members that are specific to that class (specialization).

is-a relationship

Inheritance is an **is-a relationship**. We use inheritance only if an **is-a relationship** is present between the two classes. Here are some examples:

- A car is a vehicle.
- Orange is a fruit.
- A surgeon is a doctor.
- A dog is an animal.

Real Life Example: Child inherits traits like eyes color, blood group type, hair color from parents.

Key Concepts:

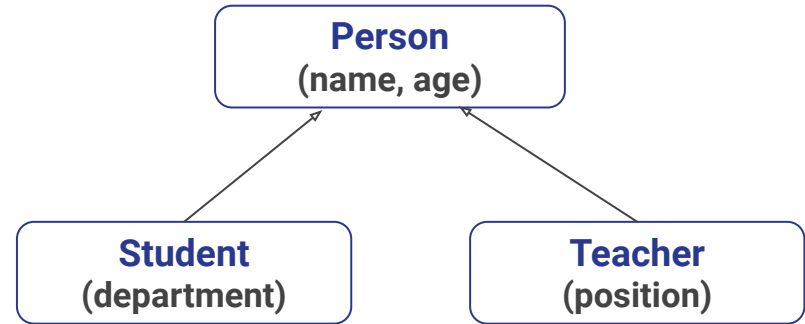
- **Base Class (parent class, superclass):** The class whose properties and methods are inherited.
- **Derived Class:(child class, subclass)** The class that inherits properties and methods from the base class.

Base Class (**Person**):

- Attributes: **name**, **age**

Derived Classes:

- **Student:** Inherits from **Person** and adds **department**.
- **Teacher:** Inherits from **Person** and adds **position**.



Example

```
//Base class
class Person{
    public:
        string name;
        int age;

        void displayPersonInfo(){
            cout<<"Name:"<< name
            <<"Age:"<< age << endl;
        }
};
```

```
//Derived class
class Student: public Person {
    public:
        string department;

        void display(){
            displayPersonInfo();
            cout<<"Department:"<< department << endl;
        }
};
```

```
//Derived class
class Teacher: public Person {
    public:
        string position;

        void display(){
            displayPersonInfo();
            cout<<"Position:"<< position << endl;
        }
};
```

Syntax

```
class DerivedClassName : access-level BaseClassName{  
    //body  
}
```

access-level (aka visibility mode, derivation type) specifies the type of derivation/inheritance, and it can be

- private (by default)
- protected or
- public

Access specifiers/modifiers

- keywords used to control the accessibility of class members
- Used extensively for hiding the data.

Types:

Private: accessible only within the class that defines them.

Public: can be accessed by any part of the program.

Protected: accessible within the class and by derived classes.

The protected access-specifier

- Protected members are like private members.
 - They are accessible by the member functions within the class but
 - are not accessible from outside the class
- The difference between private members and protected members is that unlike private members, protected members are also accessible from the derived classes.

Access specifiers	Accesible from own class	Accessible form derived class	Accessible from objects outside class
Public	Yes	Yes	Yes
Private	Yes	No	No
Protected	Yes	Yes	No

Visibility Mode: Public

When a base class is publicly inherited by a derived class,

- Private members of the base class are not inherited and therefore will never become the members of its derived class
- Public members of the base class become public members of the derived class and therefore are accessible to the objects of the derived class.
- Protected members of the base class become protected in the derived class.

```
class DerivedClassName : public BaseClassName{  
    //body  
}
```

Visibility Mode: Private

When a base class is privately inherited by a derived class,

- Private members of the base class are not inherited and therefore will never become the members of its derived class
- Public members of the base class become private members of the derived class and therefore can only be accessed by the member functions of the derived class.
- Protected members of the base class become private in the derived class.

```
class DerivedClassName : private BaseClassName{  
    //body  
}
```

Visibility Mode: Protected

When a base class is inherited in protected mode by a derived class,

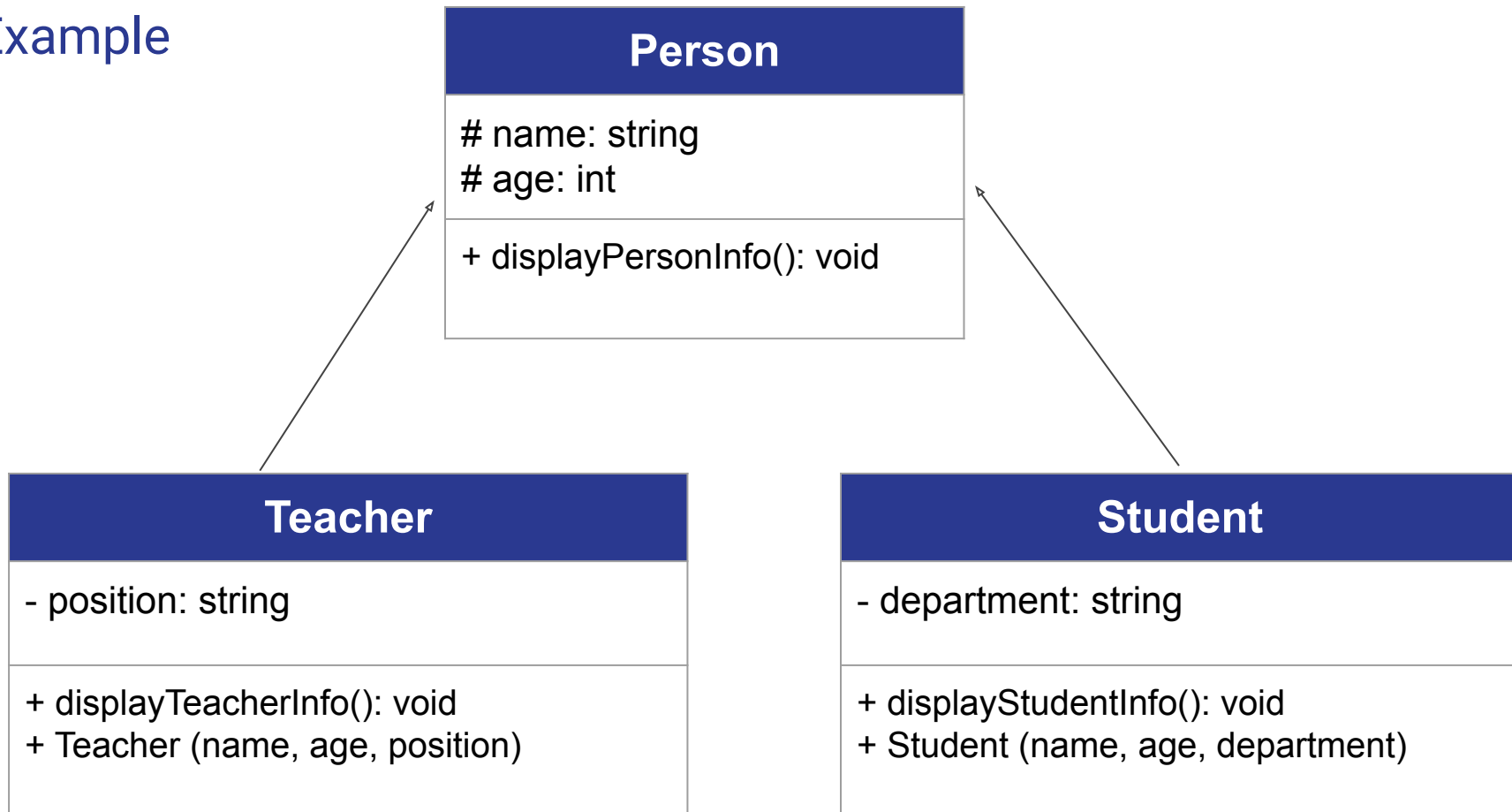
- Private members of the base class are not inherited and therefore will never become the members of its derived class
- Both public and protected members of the base class become protected members of the derived class.

```
class DerivedClassName : protected BaseClassName{  
    //body  
}
```

Access Matrix

Inherited as/ access-level	Access specifier in Base class	Access specifier in Derived class
Public <code>class Derived: public Base</code>	Public	Public
	Private	Not inherited
	Protected	Protected
Private <code>class Derived: private Base</code>	Public	Private
	Private	Not inherited
	Protected	Private
Protected <code>class Derived: protected Base</code>	Public	Protected
	Private	Not inherited
	Protected	Protected

Example



Base Class: Person

```
#include <iostream>

using namespace std;

class Person {
protected:
    string name;
    int age;
public:
    void displayPersonInfo() {
        cout<< "Name:"<< name<< "Age:" << age<< endl;
    }
};
```

Derived Class - I: Teacher

```
class Teacher : public Person {  
private:  
    string position;  
public:  
    Teacher (string n, int a, string p){  
        name = n;  
        age = a;  
        position = p;  
    }  
    void displayTeacherInfo(){  
        displayPersonInfo();  
        cout<< "Position:"<< position << endl;  
    }  
};
```

Derived Class - II: Student

```
class Student : public Person {  
private:  
    string department;  
public:  
    Student (string n, int a, string d){  
        name = n;  
        age = a;  
        department = d;  
    }  
    void displayStudentInfo() {  
        displayPersonInfo();  
        cout<< "Department:"<< department << endl;  
    }  
};
```

Main program

```
int main() {  
    Teacher t("Walter", 35, "Lecturer");  
    Student s("Pinkman", 25, "Chemistry");  
  
    cout<<"Teacher Info:"<< endl;  
    t.displayTeacherInfo();  
  
    cout<<"Student Info:"<< endl;  
    s.displayStudentInfo();  
  
    return 0;  
}
```

Types of Inheritance

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

1. Single Inheritance

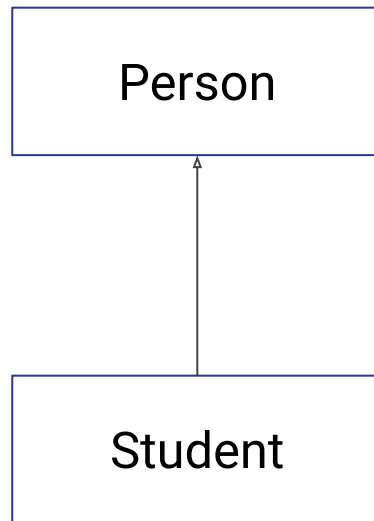
Single inheritance is when a class inherits from only one base class

A class derived from only one class.

Student is a **Person**, so class **Student** is derived from class **Person**.

```
class Person {  
    // Base class code  
};
```

```
class Student : public Person {  
    // Derived class code  
};
```



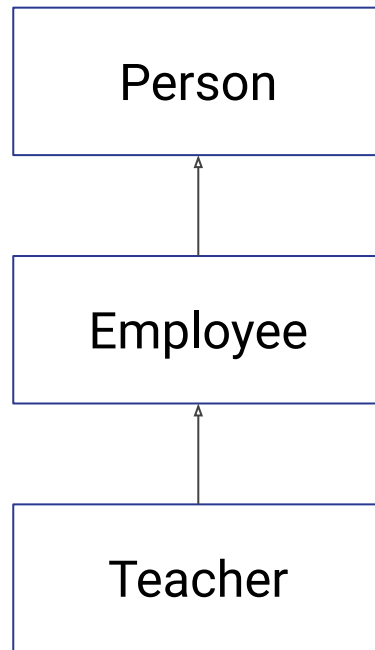
2. Multilevel Inheritance

Multilevel inheritance is when a class is derived from another derived class, creating a chain of inheritance.

A class derived from another derived class

Class **Teacher** is derived from class **Employee** and class **Employee** is derived from another class **Person**.

```
class Person {  
    // Base class code  
};  
  
class Employee : public Person {  
    // Derived class 1 code  
};  
  
class Teacher : public Employee {  
    // Derived class 2 code  
};
```



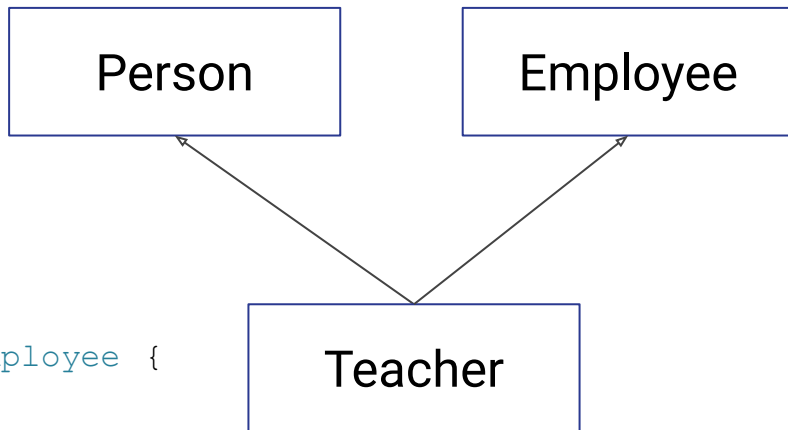
3. Multiple Inheritance

Multiple inheritance is when a class inherits from more than one base class.

A class derived from more than one base class.

A teacher is a **Person** and **Employee** as well so class **Teacher** is derived from two base classes **Person** and **Employee**.

```
class Person {  
    // Base class 1 code  
};  
class Employee {  
    // Base class 2 code  
};  
class Teacher : public Person, public Employee {  
    // Derived class code  
};
```



Problem/Ambiguity with multiple inheritance

Ambiguity arises in multiple inheritance when a derived class inherits two or more base classes that contain methods with the same name.

The derived class will have ambiguity in deciding which method to call if the methods in base classes have the same name.

Consider classes **Person** and **Employee** each containing a method named **display()**, and a class **Teacher** that inherits from both **Person** and **Employee**.

When **Teacher** tries to call **display()**, the compiler cannot determine which **display()** method to use.

```

#include <iostream>
using namespace std;
class Person{ // Base class 1
    public:
    void display(){
        cout<<"I am a person!!"<< endl;
    }
};

class Employee{ //Base class 2
    public:
    void display(){
        cout<<"I am an Employee!!"<< endl;
    }
};

// Derived class
class Teacher : public Person, public
Employee {
    //body
};

```

```

int main(){
    Teacher t;
    t.display(); // error: request for
member 'display' is ambiguous

    t.Person::display(); // Works fine
    return 0;
}

```

Multiple Inheritance

- When **t.display()** is compiled, the compiler looks to see if **Teacher** contains a function named **display()**. It doesn't.
- The compiler then looks to see if any of the parent classes have a function named **display()**.
- The problem is that object '**t**' actually contains Two **display()** functions:
 - One inherited from **Person**
 - One inherited from **Employee**
- However, there is a way to work around this problem. We can explicitly specify which version we meant to call.

Example: `t.Person::display()` //call Person version of display()

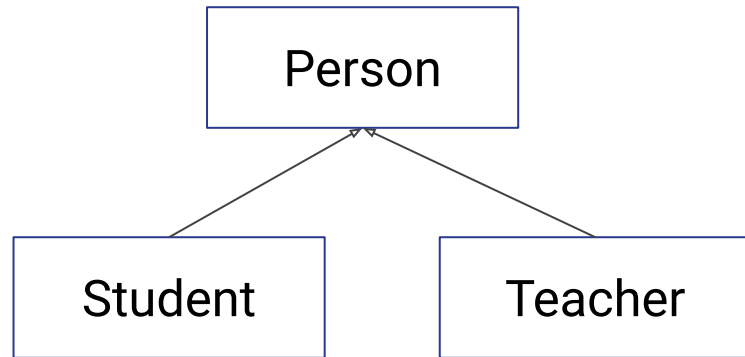
4. Hierarchical Inheritance

Hierarchical inheritance is when multiple classes inherit from a single base class.

More than one class are derived from Single Base class.

Teacher and **Student** all are **Person**, so class **Student** and **Teacher** are derived from class **Person**.

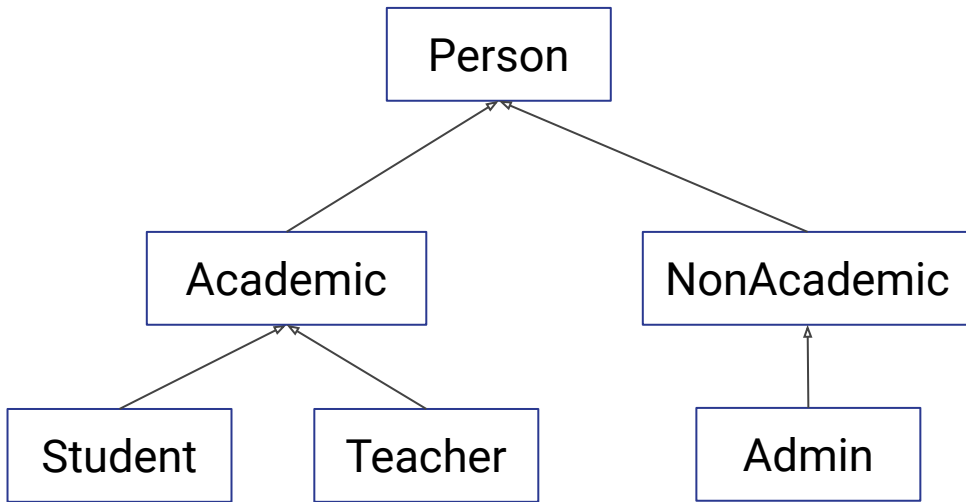
```
class Person {  
    // Base class code  
};  
class Student : public Person {  
    // Derived class 1 code  
};  
class Teacher : public Person {  
    // Derived class 2 code  
};
```



5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.

```
class Person {  
    // Base class code  
};  
class Academic : public Person {  
    // Derived class code  
};  
class NonAcademic : public Person {  
    // Derived class code  
};  
class Student : public Academic {  
    // Derived class code  
};  
class Teacher : public Academic {  
    // Derived class code  
};  
class Admin : public NonAcademic {  
    // Derived class code  
};
```

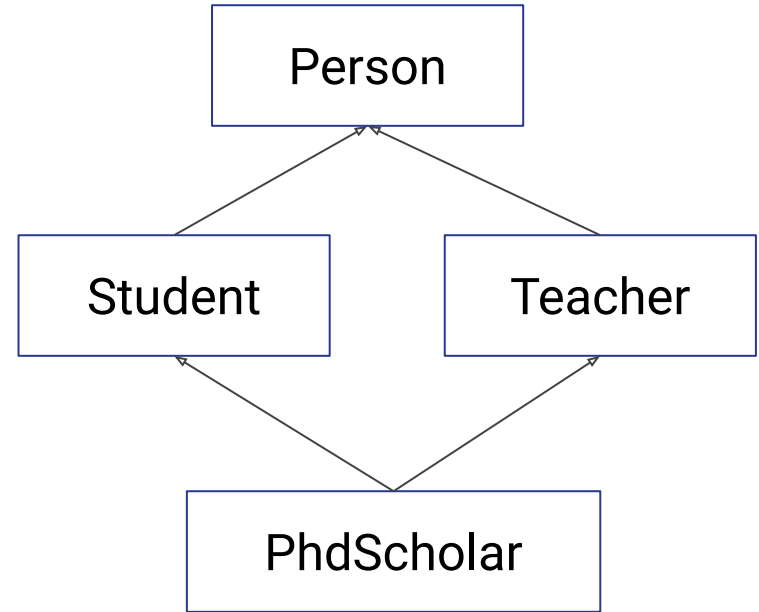


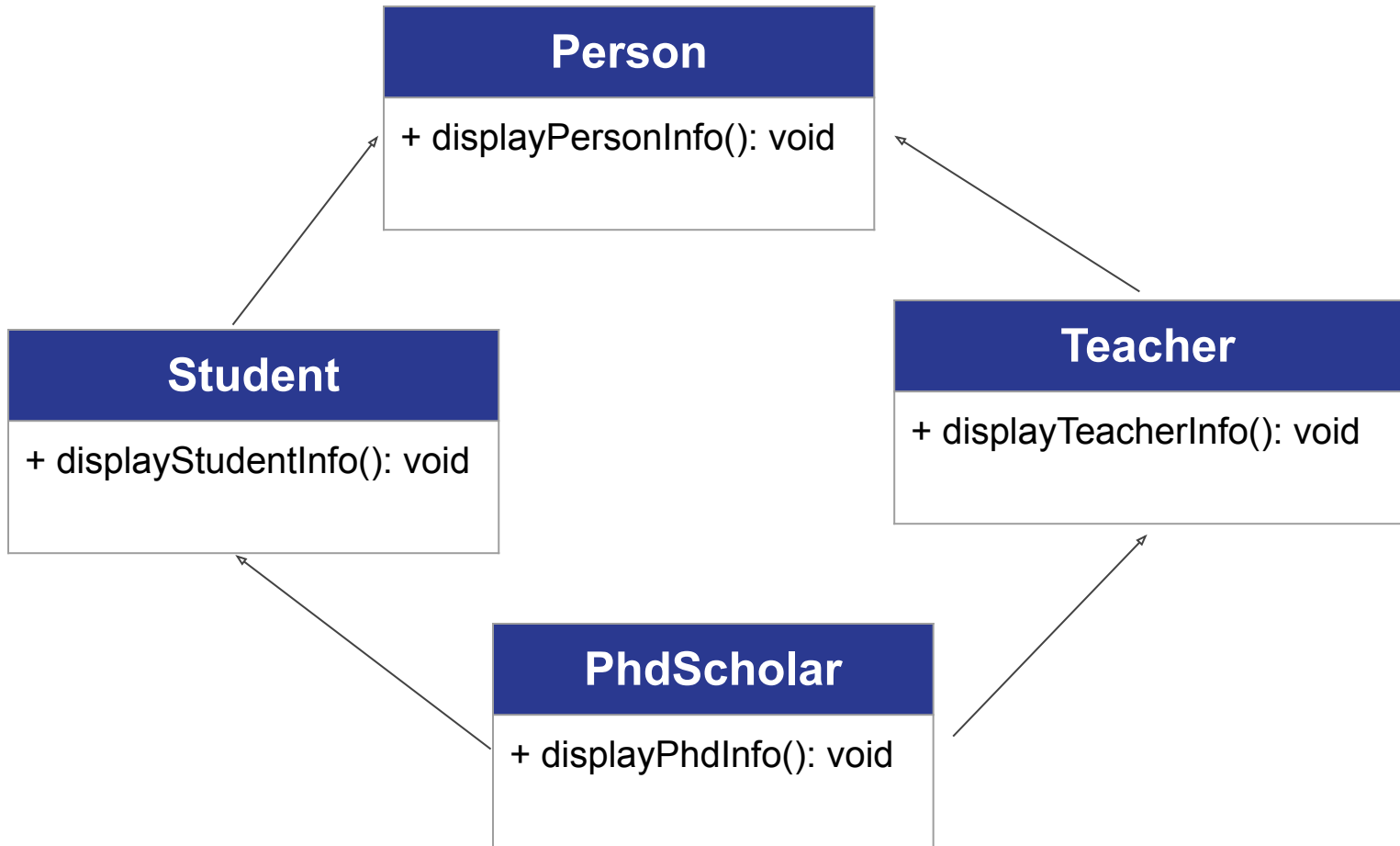
The Diamond Problem

The diamond problem occurs when two or more derived classes inherit from the same base class, and another class inherits from these derived classes, creating an ambiguity.

Here, **PhdScholar** class would get the two copies of **Person** class. One from **Student** and one from **Teacher**.

When any data/member function of class **Person** is accessed by an object of class **PhdScholar**, ambiguity arises as which data/member function would be called.





```

#include <iostream>
using namespace std;
class Person {
public:
    void displayPersonInfo () {
        cout << "I am a Person!!" << endl;
    }
};
class Student : public Person {
public:
    void displayStudentInfo () {
        cout << "I am a Student!!" << endl;
    }
};
class Teacher : public Person {
public:
    void displayTeacherInfo () {
        cout << "I am a Teacher!!" << endl;
    }
};

```

```

class PhdScholar : public Student, public
Teacher {
public:
    void displayPhdInfo () {
        cout << "I am a PhD Scholar!!" << endl;
    }
};

int main() {
    PhdScholar phd;
    phd.displayPhdInfo ();

    // Ambiguous call: Which displayPerson() to
    use?
    phd.displayPersonInfo (); // Error: request
    for member 'displayPerson' is ambiguous

    return 0;
}

```

Virtual Base Class

To resolve this ambiguity (diamond problem), common base class has to be declared as virtual base class by inserting the “**virtual**” keyword in the inheritance list of the derived class.

Virtual Base Class means there is only one base object.

The base object is shared between all objects in the inheritance tree and is only constructed once.

Virtual base classes ensure that only one instance of the base class is shared among all derived classes, preventing ambiguity and redundancy.

In case of our example,

Approach: Use virtual inheritance to ensure that only one instance of **Person** is inherited by **PhdScholar**.

Modification: Change **Student** and **Teacher** to virtually inherit from **Person**.

```

#include <iostream>
using namespace std;
class Person {
public:
    void displayPersonInfo() {
        cout << "I am a Person!!" << endl;
    }
};

class Student : virtual public Person {
public:
    void displayStudentInfo() {
        cout << "I am a Student!!" << endl;
    }
};

class Teacher : virtual public Person {
public:
    void displayTeacherInfo() {
        cout << "I am a Teacher!!" << endl;
    }
};

```

```

class PhdScholar : public Student, public
Teacher {
public:
    void displayPhdInfo () {
        cout << "I am a PhD Scholar!!" << endl;
    }
};

int main() {
    PhdScholar phd;
    phd.displayPhdInfo ();

    // No ambiguity: Single instance of Person
    phd.displayPersonInfo (); // Now this works
    correctly

    return 0;
}

```

Function Overriding

Definition: When the base class and derived class have member functions with exactly the same name, same return-type, and same arguments list, then it is said to be function overriding.

Purpose: Allows a derived class to offer a specialized behavior for a method that is already provided by its base class.

Overriding is needed when derived class function has to do some added or different job than the base class function.

It is used to achieve runtime polymorphism.

```
#include <iostream>
using namespace std;
class Person {
public:
    void display() {
        cout << "I am a Person!" << endl;
    }
};

class Teacher : public Person {
public:
    void display() {
        cout << "I am a Teacher!" <<
endl;
    }
};
```

```
int main() {
    Person p;
    Teacher t;

    p.display(); // Outputs: I am a Person!
    t.display(); // Outputs: I am a Teacher!
    t.Person::display(); // Outputs: I am a
Person!

    return 0;
}
```

Output:

```
I am a Person!
I am a Teacher!
I am a Person!
```

Feature	Function Overloading	Function Overriding
Definition	Multiple functions with the same name but different parameters or types within the same scope (class or namespace).	Redefining a base class function in a derived class with the same name, return type, and parameters.
Inheritance Requirement	Not required to involve inheritance.	Requires inheritance where a base class function is overridden in a derived class.
Base Function	Does not involve a base function; each overloaded function is independent.	Requires a base function in the base class that is overridden in the derived class.
Signature	Must differ in number or types of parameters (or both).	Must have the same name, return type, and parameters as the base class function.
Static vs Dynamic Binding	Resolved at compile-time (static polymorphism).	Resolved at runtime (dynamic polymorphism), requires virtual function.

Abstract Class

- An **abstract** class is one that is **not** used to create **objects**.
- It is designed only to act as a **base** class.
- Abstract class cannot be **instantiated** but pointers and reference of abstract class type can be created.
- A class is **Abstract** class when it has **at least one pure virtual function**.
- **Pure virtual function** simply acts as a placeholder that is meant to be redefined by derived classes. A pure virtual function has no **body** at all.
- To create a pure virtual function rather than define a body for the function, we simply assign the **function the value 0**.
- Pure virtual function must be overridden by derived classes.

```

class Person {
public:
    string name;
    int age;
    virtual void display() = 0; // Pure virtual
function
};

class Student : public Person {
public:
    string depart;
    // constructor to initialize values
    Student (string n, int a, string d){
        name = n;
        age = a;
        depart = d;
    }
    // Overriding pure virtual function is must
    void display(){
        cout << "Name: " << name << "Age:" << age <<
"Department:" << depart << endl;
    }
};

```

```

int main() {
    // Person p; // Error: cannot declare
variable 'p' to be of abstract type
'Person'
    Student s("Pinkman", 20, "Chemistry");

    s.display();

    return 0;
}

```

- **Person** class has a **pure virtual function display()**, making it **abstract**. It cannot create **objects**.
- **Student** class inherits from **Person** and **compulsory** to **override** the display() function.
- **Virtual** functions helps to achieve **polymorphism**.

Constructor and Destructor

Constructor: A special member function that is called when an object is created.

Destructor: A special member function that is called when an object is destroyed.

Purpose: Constructors initialize objects, and destructors clean up resources.

Order of Construction of derived class

In a derived class, the base class constructor is called first, followed by the derived class constructor.

The base class part of the derived class needs to be initialized before the derived class itself.

In the case of the default constructor, it is implicitly available from Parent to Child Class.

Parameterized constructor are not accessible to the derived class automatically.

Explicit call has to be made in the child class constructor for accessing the parameterized constructor of the Parent Class to the Child Class.

```

class Person {
public:
    string name;

    Person() {
        cout << "Person default constructor called" << endl;
    }
    ~Person() {
        cout << "Person destructor called for " << name << endl;
    }
};

class Student : public Person {
public:
    string depart;

    Student(string n, string d) {
        name = n;
        depart = d;
        cout << "Student constructor called for " << name << endl;
    }
    ~Student() {
        cout << "Student destructor called for " << name << endl;
    }
};

```

```

int main() {
    Student s("Pinkman",
             "Chemistry");

    return 0;
}

```

Output:

Person default constructor called
 Student constructor called for Pinkman
 Student destructor called for Pinkman
 Person destructor called for Pinkman

Note: Order of destructor is in reverse order of the constructor

Order of construction of derived class

Order of Construction for Inheritance Chain

It is sometimes the case that classes are derived from other classes, which are themselves derived from other classes.

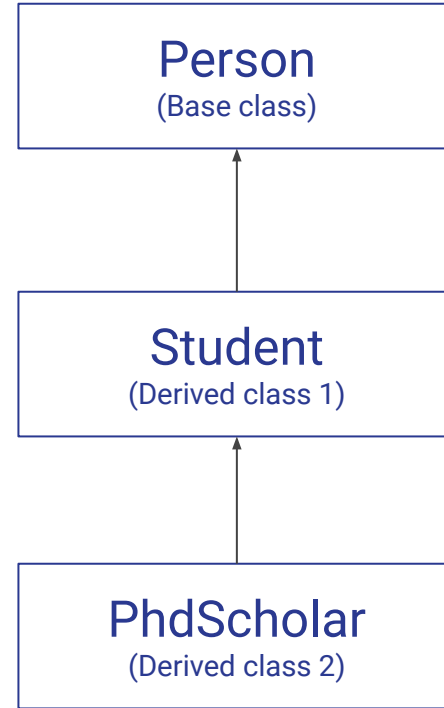
C++ always constructs the “first” or “most base ” class first. It then walks through the inheritance tree inorder and constructs each successive derived class.

Note: Order of destructor is in reverse order of the constructor

Example:

Multi-level inheritance

What will be the order of construction and destruction if we create an instance (object) of PhdScholar?



```

class Person {
public:
    Person() {
        cout << "Person constructor called" <<
endl;
    }
    ~Person() {
        cout << "Person destructor called" << endl;
    }
};

class Student : public Person {
public:
    Student() {
        cout << "Student constructor called" <<
endl;
    }
    ~Student() {
        cout << "Student destructor called" <<
endl;
    }
};

```

```

class PhdScholar : public Student {
public:
    PhdScholar() {
        cout << "PhdScholar constructor called"
<< endl;
    }
    ~PhdScholar() {
        cout << "PhdScholar destructor
called" << endl;
    }
};

int main() {
    PhdScholar p;
    return 0;
}

```

Output:

```

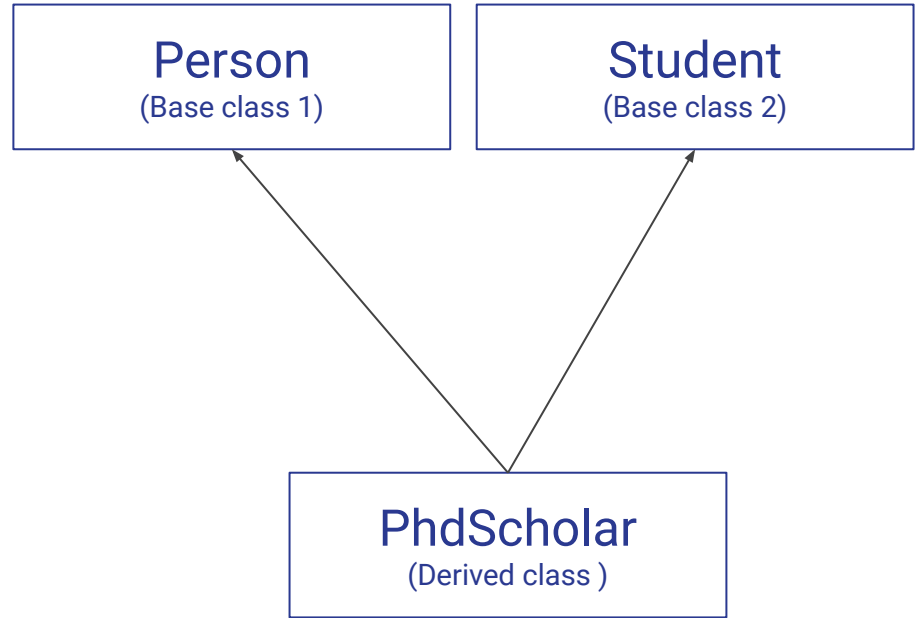
Person constructor called
Student constructor called
PhdScholar constructor called
PhdScholar destructor called
Student destructor called
Person destructor called

```

Example:

Multiple inheritance

What will be the order of construction and destruction if we create an instance (object) of PhdScholar?



```

class Person {
public:
    Person() {
        cout << "Person constructor called" <<
endl;
    }
    ~Person() {
        cout << "Person destructor called" << endl;
    }
};

class Student{
public:
    Student() {
        cout << "Student constructor called" <<
endl;
    }
    ~Student() {
        cout << "Student destructor called" <<
endl;
    }
};

```

```

class PhdScholar : public Person, public
Student {
public:
    PhdScholar() {
        cout << "PhdScholar constructor
called" << endl;
    }
    ~PhdScholar() {
        cout << "PhdScholar destructor called"
<< endl;
    }
};

int main() {
    PhdScholar p;
    return 0;
}

```

Output:

```

Person constructor called
Student constructor called
PhdScholar constructor called
PhdScholar destructor called
Student destructor called
Person destructor called

```

Try it!! - Diamond Problem

A class "**master**" derives information from "**account**" and "**admin**" classes which in turn are derived from the class "**person**". Define all the four classes and write a program to create, update and display the information contained in master objects using concept of virtual base class. (2018)

person has data members: **name** and **code**

account has data member: **pay**

admin has data member: **experience**

master has derived data members: **name**, **code**, **experience** and **pay**



Any Queries?